

# 50 System Design Interview Questions: Tadka, Weeks 1 to 4

*Scenario-based questions and detailed answers covering everything through Day 8: monolith foundations, domain modeling, API hardening, scaling, and the first service extraction*

## Contents

How to use this document

### Part 1: Week 1, Architect the Monolith, Foundation and Domain Modeling

- Q1. Why start Tadka as a monolith instead of microservices?
- Q2. What is an ADR, and why write one instead of deciding in a chat message?
- Q3. Applying the four-step decision framework to a real FR/NFR split
- Q4. How do you decide domain boundaries inside one codebase?
- Q5. What does schema-per-domain actually look like in one Postgres instance?
- Q6. Should Orders have a foreign key directly into Payment's tables?
- Q7. How do you defend "1 lakh orders/day" as a scale target?
- Q8. Separating functional from non-functional requirements
- Q9. "Tadka runs on your laptop", why is that a legitimate architecture statement?
- Q10. Interviewer says "design a food delivery app", what do you ask first?
- Q11. When do you actually flip from monolith to microservices?
- Q12. Choosing a primary key strategy for Orders up front

### Part 2: Week 2, Monolith API, State Machine and Hardening

- Q13. Why use one consistent error response shape across every endpoint?
- Q14. When do you return 400, 422, or 404?
- Q15. Why must order price always be computed server-side?
- Q16. Designing the order state machine
- Q17. The double-tap problem
- Q18. The lost update problem
- Q19. BookMyShow-style oversell, solved with Postgres `xmin`

- Q20. Ordering a side effect (SMS) relative to a database commit
- Q21. Why should the API never leak internal exception details?
- Q22. Where should "restaurant is closed" be validated?
- Q23. Implementing an idempotency key end to end
- Q24. Why does a resource need a stable, immutable identifier?
- Q25. An order stuck forever in "PaymentPending"

### **Part 3: Week 3, Architect for Scale, Scaling the Monolith**

- Q26. Query latency degrades from 50ms to 3 seconds as the table grows
- Q27. Why doesn't adding more app servers fix a database bottleneck?
- Q28. The cache-aside pattern for the menu API, end to end
- Q29. "The cache lies": stale menu prices for ten minutes
- Q30. Live order tracking without polling
- Q31. The hot key problem: a restaurant goes viral
- Q32. Cache stampede at exactly 12:00 noon
- Q33. Connection pool exhaustion during a flash sale
- Q34. When do you actually need a read replica?
- Q35. Redis is down for 30 minutes during peak
- Q36. Defending a scale number before choosing Redis, replicas, or sharding
- Q37. Rate limiting the public order-placement endpoint
- Q38. Why Redis before splitting into microservices?

### **Part 4: Week 4, Modular Monolith and First Service Extraction**

- Q39. What does a module boundary actually look like in code?
- Q40. The four questions before extracting Restaurant into its own service
- Q41. Why Payment first, not Orders or Restaurant?
- Q42. Proving fault isolation after extracting Payment
- Q43. Why HTTP now, Kafka deliberately later
- Q44. What PCI compliance actually requires, versus what a service boundary provides
- Q45. Why sharing one Postgres instance between Order and Payment breaks the boundary
- Q46. WAL and crash recovery: does a confirmed order survive a mid-transaction crash?
- Q47. "Order not found" immediately after placing it
- Q48. Two minutes of confirmed orders missing after a failover

Q49. Scaling to 50 read replicas: direct fan-out versus cascading

Q50. The durability wound: a lost in-memory payment job after extraction

Closing note

---

## How to use this document

Each question below is written the way it would actually arrive in a system design interview: as a scenario with real numbers, not an abstract definition to recite. Every answer is structured the same way:

- **Scenario:** the situation, with concrete numbers wherever numbers matter.
- **Question:** what the interviewer is actually asking.
- **Answer:** the reasoning an architect-level candidate would give, in full.
- **Key principle:** the one sentence worth remembering, even if you forget everything else about this question.

Read a question, try to answer it yourself in two or three sentences before reading the model answer. That is the only way this kind of preparation actually works. Reading fifty answers passively will not make you sound like an architect in a real interview; working through the reasoning yourself, then checking it against these answers, will.

The document is organized to match the four weeks it is built on: foundations and domain modeling, API and state machine hardening, scaling the monolith, and the first real service extraction.

# Part 1

## *Week 1, Architect the Monolith, Foundation and Domain Modeling*

### Q1. Why start Tadka as a monolith instead of microservices?

**Scenario.** A founder tells you, "we're building the next big food delivery platform, let's design it with microservices from day one so we don't have to rewrite anything later."

**Question.** Do you agree, and why or why not?

**Answer.** No, and this is worth pushing back on directly. Microservices solve organizational and operational problems: independent deployment for separate teams, independent scaling for genuinely different load profiles, and fault isolation between capabilities that must not share fate. On day one, Tadka has none of these problems, because it has one team, one deployment cadence, and no evidence yet about which capability will need to scale differently from the others. Starting with microservices means paying the full cost of a distributed system, network calls, service discovery, distributed debugging, eventual consistency, before there is a single real requirement that justifies it. A well-structured monolith, with clean internal module boundaries (covered in Week 4), is not a stepping stone you are embarrassed about; it is the correct architecture for the actual constraints on day one, and it keeps the door open to extract a service later, for a specific, evidenced reason, exactly the way Payment was eventually extracted.

**Key principle.** *Architecture should match today's actual team size, traffic, and failure requirements, not tomorrow's imagined ones.*

### Q2. What is an ADR, and why write one instead of deciding in a chat message?

**Scenario.** Your team debates in a Slack thread whether Tadka should use REST or GraphQL for its public API, someone says "let's just go with REST," and everyone moves on.

**Question.** What is missing from this process, and what would you do instead?

**Answer.** What is missing is a durable, structured record of *why* the decision was made, which becomes critical the day someone new joins the team, or the day the decision needs to be revisited under different circumstances. An Architecture Decision Record (ADR) captures four things explicitly: the context (what problem existed, what constraints applied at the time), the decision itself (what was actually chosen), the consequences (what this buys you and what it costs, both positive and negative), and, ideally, the alternatives that were considered and why they were rejected. Writing "we chose REST over GraphQL because our clients are simple CRUD-style mobile apps, our team has no GraphQL experience, and our query patterns are well known and stable" is a completely different, more valuable artifact than "let's just go with REST" buried in a chat thread that nobody will find in eight months. ADRs are not bureaucracy for its own sake; they are how an architecture explains itself to the people who inherit it.

**Key principle.** *A decision without a written reason is a decision nobody can safely revisit later.*

### Q3. Applying the four-step decision framework to a real FR/NFR split

**Scenario.** Product wants Tadka's restaurant search to support "find restaurants serving biryani near me, sorted by rating, open now." Someone proposes Elasticsearch immediately.

**Question.** Walk through how you would actually decide between Elasticsearch and PostgreSQL's built-in full-text and geo search for this feature.

**Answer.** Apply the same four-step reasoning used for every architectural decision in this course: name the problem precisely, check whether it exists today with evidence, check whether the simpler option can solve it, and name the new failure modes the more complex option introduces. The functional requirement is "text search plus geo-distance filtering plus sorting." The non-functional requirements that actually matter are the current restaurant count (if it is in the thousands, not millions, this is not a search-scale problem yet), the acceptable query latency, and whether the team has any existing Elasticsearch operational experience. PostgreSQL's `tsvector` full-text search combined with the PostGIS extension for geo-distance can comfortably serve tens of thousands of restaurants with sub-100ms queries, using infrastructure the team already runs and understands. Elasticsearch becomes justified once search relevance tuning, fuzzy matching at scale, or genuinely large restaurant volume become real, evidenced requirements, not because "Elasticsearch is what real search uses."

**Key principle.** *Match the tool to the current, evidenced requirement, not to the tool's reputation for the problem in general.*

## Q4. How do you decide domain boundaries inside one codebase?

**Scenario.** Two engineers disagree: one wants `DeliveryPartner` to live inside the `Order` domain since delivery assignment happens as part of order fulfillment; the other wants it as its own domain.

**Question.** How do you settle this, in general terms that apply beyond just this one case?

**Answer.** A domain boundary should track things that change for the same reason and are owned by the same concept, not things that merely interact frequently. `Order` and `DeliveryPartner` interact constantly, an order needs a delivery partner assigned, but they change for entirely different reasons: `Order` changes because of business rules about pricing, discounts, and order lifecycle; `DeliveryPartner` changes because of onboarding rules, availability tracking, ratings, and payout logic, none of which have anything to do with a specific order's lifecycle. The test worth applying explicitly: if you can imagine a delivery partner existing, being rated, and being paid completely independently of any specific order's existence, it is a separate domain that an order merely *references*, the same way `Order` references a `Restaurant` without owning restaurant logic. The practical structure is `Order` holding a `deliveryPartnerId` as a reference, with delivery-partner-specific logic living entirely inside its own domain module.

**Key principle.** *Two concepts that interact often are not automatically the same domain; ask what makes each one change, independently of the other.*

## Q5. What does schema-per-domain actually look like in one Postgres instance?

**Scenario.** Tadka's monolith has `Order`, `Payment`, `Restaurant`, and `User` domains, all currently living in Postgres's default `public` schema, with table names like `orders`, `payments`, `restaurants`.

**Question.** What is the problem with this, and what would you change?

**Answer.** With everything in one flat `public` schema, nothing in the database itself signals which tables belong to which domain, and nothing prevents `OrderService` code from directly querying `payments` tables just because they happen to be reachable in the same connection. The fix is creating a dedicated Postgres schema per domain, `orders.orders`, `payments.transactions`, `restaurants.menus`, `users.accounts`, and configuring each domain's code to connect using a role that only has privileges on its own schema. This gives you a real, database-enforced boundary, not just a naming convention: `OrderService`'s database role genuinely cannot query the `payments` schema, even by accident, even under deadline pressure, because PostgreSQL itself will reject the query with a permissions error. This is the single-database equivalent of the "architecture test that fails the build" idea applied at the data layer instead of the code layer, and it is also exactly what makes a later extraction into `Payment`'s own physical database a mechanical move rather than an untangling exercise.

**Key principle.** *A domain boundary enforced by database permissions survives deadline pressure in a way a naming convention alone does not.*

## Q6. Should Orders have a foreign key directly into Payment's tables?

**Scenario.** A developer proposes adding `orders.payment_id` as a proper foreign key constraint referencing `payments.transactions.id`, "to guarantee referential integrity."

**Question.** Is this a good idea, given the schema-per-domain boundary from the previous question?

**Answer.** No, and this is a subtle but important trap. A database-enforced foreign key across schema boundaries recreates exactly the coupling schema-per-domain was meant to prevent: it means `Payment`'s table structure can never change (a column rename, a primary key type change, a future migration to a separate physical database) without `Order`'s schema also needing a matching migration, because the constraint physically ties them together. The correct pattern is a **logical reference**: `orders.orders` stores a `payment_id` as a plain column, with no database-level foreign key constraint enforcing it, and referential correctness is maintained at the application layer instead, through the code that creates orders and payments together. This costs you the database's automatic integrity checking for this one relationship, which is a real, deliberate trade-off, made specifically because this is a domain boundary, not an ordinary internal relationship within a single domain's own tables, where foreign keys remain exactly the right tool.

**Key principle.** *A foreign key across a domain boundary is a hidden coupling; keep the reference, drop the database-enforced constraint.*

## Q7. How do you defend "1 lakh orders/day" as a scale target?

**Scenario.** An interviewer asks, "why did you design for 1 lakh orders a day specifically, and not 10 lakh, or 10,000?"

**Question.** How do you answer this without sounding like you picked a random round number?

**Answer.** The right answer works backward from the number to its consequences, showing you understand what the number actually implies, not just that you memorized it. One lakh orders a day averages to roughly 1.2 orders per second, but real traffic is never flat; a typical dinner-rush peak might run at 15 to 20 times the average for a few concentrated hours, putting peak load in the range of 20 to 25 orders per second. That peak number is what actually drives capacity decisions: it tells you whether a single Postgres instance can handle it (it comfortably can, at this scale, with reasonable indexing),

whether you need a cache in front of read-heavy endpoints like the menu (yes, because menu reads vastly outnumber order writes), and whether you need to split services for load reasons yet (no, not at this volume). The number is defensible not because it is realistic for a specific company, but because you can show the arithmetic that turns "orders per day" into "peak requests per second," and connect that peak number to specific, concrete infrastructure decisions.

**Key principle.** *A scale number is only useful once you have converted it into a peak-load figure and connected that figure to an actual infrastructure decision.*

## Q8. Separating functional from non-functional requirements

**Scenario.** Product writes a one-line requirement: "customers should be able to pay cash on delivery."

**Question.** How do you turn this into an actual specification, distinguishing functional from non-functional requirements?

**Answer.** The functional requirement is narrow and concrete: the order-placement flow must accept a `paymentMethod` of `CASH_ON_DELIVERY`, skip the online payment gateway call entirely, and mark the order for collection at delivery time. Everything else worth asking about is a non-functional requirement hiding inside that one sentence: what happens if the delivery partner cannot collect the exact cash amount (a change-management NFR), how is a cash order reconciled against the restaurant's payout at the end of the day (a data-consistency and reporting NFR), what fraud or abuse controls exist against customers who repeatedly refuse cash-on-delivery orders at the door (a business-risk NFR), and whether cash-on-delivery is available for every order value or capped below some threshold (a policy NFR that directly affects the state machine's design). A junior engineer implements the one-line functional requirement. An architect surfaces the four or five non-functional questions hiding underneath it before writing any code, because each one changes the design.

**Key principle.** *Every short functional requirement has non-functional requirements hiding inside it; find them before you design, not after a production incident finds them for you.*

## Q9. "Tadka runs on your laptop", why is that a legitimate architecture statement?

**Scenario.** A student, mid-course, jokes that "Tadka is a toy project, real companies would never actually run like this."

**Question.** Is the reduced infrastructure a weakness of the teaching example, or is there a real architectural point being made?

**Answer.** There is a completely real point, and it is one of the most commonly violated principles in the industry: infrastructure should be sized to the *actual current load*, not to an imagined future load or to what "real companies" are assumed to run. At Tadka's teaching scale, a single Postgres instance, a single Redis instance, and one application process genuinely can handle the described order volume with room to spare, and building a Kubernetes cluster with twelve microservices for that same load would be pure premature engineering, adding operational cost and complexity that buys nothing, because the problems that complexity solves (independent team deployment, capacity for genuinely different scaling profiles) do not exist yet. The lesson generalizes far past Tadka: a huge fraction of real production systems are over-architected for a scale they will never actually reach, at real, ongoing cost in complexity and hiring difficulty, precisely because nobody was willing to say out loud that the simple

version would have been enough.

**Key principle.** *Provision and design for the load you actually have, evidenced today, not the load you imagine you might someday need.*

## Q10. Interviewer says "design a food delivery app", what do you ask first?

**Scenario.** You are handed the open-ended prompt "design a system like Swiggy or Zomato" with no further detail.

**Question.** What are the first questions you ask, before drawing a single box?

**Answer.** Resist the urge to start drawing architecture immediately; the candidates who stand out are the ones who scope the problem first. Ask about scale (how many orders per day, how many restaurants, how many concurrent users during peak), ask about which specific capability the interview wants to focus on (the full platform is enormous; most interviews want depth on one or two slices, like order placement and live tracking, not shallow coverage of everything), ask about consistency and correctness requirements (can a customer ever be double-charged, must the system never oversell a promotion), and ask what already exists versus what you are designing from scratch (are you designing a brand-new system, or evolving an existing one, which changes whether "should we split into microservices" is even a live question). An architect-level answer to an open-ended prompt looks like five minutes of scoping questions before a single line is drawn, not a whiteboard immediately filling up with boxes.

**Key principle.** *An unscoped system-design prompt is itself a test of whether you ask before you architect.*

## Q11. When do you actually flip from monolith to microservices?

**Scenario.** A stakeholder asks, "so at what point, concretely, do we switch from the monolith to microservices?"

**Question.** Give a concrete, non-hand-wavy answer.

**Answer.** The honest answer is that there is no single global trigger; the decision has to be made module by module, driven by evidence for that specific module, not as a wholesale platform migration on a calendar date. The concrete triggers, each independently sufficient, are: a specific module's traffic profile is measurably and persistently different from the rest of the system (justifying independent scaling), a specific module handles data with materially stricter security or compliance requirements than the rest (justifying a stronger boundary), a specific module is owned by a genuinely separate team with a different release cadence that is currently blocked by shared deployment, or a specific module's failure mode needs to be isolated from the rest of the system for a real, named reason. Tadka's actual path illustrates this precisely: Payment was extracted first, specifically for fault isolation and a stronger security boundary, while Orders and Restaurant stayed inside the monolith, because neither of them had yet accumulated the same evidence.

**Key principle.** *"When should we go microservices" is the wrong question; ask "which specific module has evidence justifying extraction," one module at a time.*

## Q12. Choosing a primary key strategy for Orders up front

**Scenario.** At domain-modeling time, before any code is written, a teammate asks whether the orders table's primary key should be an auto-incrementing integer or a UUID.

**Question.** What do you decide, and what future consideration should influence the decision now?

**Answer.** For Tadka's current scale, a sequential BIGINT primary key is the better default: it keeps the underlying B-tree index compact and fast to insert into, since new rows are always appended near the end of the index rather than scattered randomly across it, which is exactly what a random UUID would cause. The forward-looking consideration worth raising now, even though it will not be acted on yet, is that if Orders is ever sharded across multiple database instances (a Week 3-and-beyond scaling concern), a plain auto-incrementing integer stops being safely unique across shards, and the identifier strategy would need to change to something shard-aware, a UUID, or a sequential-but-still-globally-unique scheme like UUIDv7, or an ID-generation service. The right move today is not to prematurely adopt a UUID "just in case," which pays the locality cost now for a sharding requirement that does not exist yet; it is to keep the customer-facing order reference (whatever appears in the app and receipts) decoupled from the internal primary key, so that the internal key can be changed later without breaking anything customer-facing.

**Key principle.** *Decouple the externally visible identifier from the internal primary key, so the internal key can evolve later without an external migration.*

## Part 2

### Week 2, Monolith API, State Machine and Hardening

#### Q13. Why use one consistent error response shape across every endpoint?

**Scenario.** Tadka's `POST /orders` returns `{"error": "Invalid restaurant ID"}` on failure, while `POST /payments` returns `{"message": "Payment failed", "code": 42}`, a different shape entirely.

**Question.** Why is this a problem, and what should the fix look like?

**Answer.** Every client integrating with Tadka's API, the mobile app, a partner integration, an internal admin tool, now has to write separate, bespoke error-parsing logic for every single endpoint, because there is no predictable shape to rely on. The fix is a single, consistent error envelope used everywhere, commonly following something close to the RFC 7807 "Problem Details" convention: a `type` identifying the error category, a `title` giving a short human-readable summary, a `status` matching the HTTP status code, a `detail` giving specifics, and an optional `instance` or `traceId` for support and debugging. Every endpoint in the system, regardless of which domain it belongs to, returns errors in exactly this shape, so any client can write one error-handling code path that works everywhere, and so any new endpoint added six months from now automatically fits the same contract without a design discussion.

**Key principle.** *A consistent error contract is an API design decision made once, for the whole system, not an implementation detail decided per endpoint.*

#### Q14. When do you return 400, 422, or 404?

**Scenario.** Three different `POST /orders` requests fail: one sends malformed JSON, one sends valid JSON but with a `restaurantId` that does not exist, and one sends valid JSON with a quantity of `-5`.

**Question.** What status code does each one get, and why does the distinction matter?

**Answer.** Malformed JSON that the server cannot even parse is a `400 Bad Request`, the most generic "the request itself is broken" signal. A `restaurantId` that does not exist is closer to a `404 Not Found` in spirit, though many APIs deliberately return this as part of a `422` validation response instead, since the request body was syntactically and structurally valid, it just referenced something that failed a business-level check; the distinction is worth a deliberate team decision, documented once, not decided ad hoc per endpoint. A negative quantity is a textbook `422 Unprocessable Entity`: the JSON is syntactically valid and well-formed, but it violates a business rule the domain itself defines. The distinction matters because a client's retry logic depends on it: a `400` typically means "fix your request and don't retry the same payload," a `422` means "this specific business rule failed, show the user why," and a `404` means "this resource genuinely does not exist." Collapsing all three into a generic `500` or an inconsistent `400` for everything actively breaks client-side error handling.

**Key principle.** *The status code is part of the contract the client programs against; picking the wrong one breaks client logic even when the error message text is perfectly clear to a human.*

#### Q15. Why must order price always be computed server-side?

**Scenario.** A mobile app sends `POST /orders` with a body that includes `{"items": [...], "totalPrice": 249}`, and the server saves that price directly.

**Question.** What is wrong with this design, and what would you change?

**Answer.** Trusting a client-supplied price is a direct, exploitable vulnerability: nothing stops a modified client, or a raw API call crafted with a tool like Postman, from sending `{"items": [{"itemId": "expensive-biryani"}], "totalPrice": 1}`, and if the server saves that price as-is, the restaurant and the platform lose money on every single order placed this way. The fix is that the server must independently recompute the price from its own source of truth every single time: look up each item's current price from the database, apply any valid discounts or promotions server-side, compute taxes and delivery fees using server-side logic, and entirely ignore any price field the client happens to send. The general principle this illustrates is broader than pricing alone: **the client can be trusted to describe intent** ("I want two orders of this item"), **never to describe the consequence** ("and it costs this much"), because the consequence is exactly the part an attacker is most motivated to falsify.

**Key principle.** *Never trust a client to tell the server what something costs; the server always computes cost from its own data.*

## Q16. Designing the order state machine

**Scenario.** Tadka's orders currently have a single `status` string field that any code path can set to any value at any time, and a bug recently allowed an order to go from `Delivered` back to `Created`.

**Question.** How would you redesign this to make that bug structurally impossible?

**Answer.** Define the order lifecycle as an explicit state machine with a fixed, enumerated set of states, `Created`, `PaymentPending`, `Confirmed`, `Preparing`, `OutForDelivery`, `Delivered`, `Cancelled`, and, separately, an explicit table or in-code map of exactly which transitions are legal from each state. `Delivered` should have no outgoing transitions defined at all except perhaps to a `Refunded` state under a very specific, separately authorized flow; `Created` → `Delivered` directly should never appear in the legal-transition map. The enforcement point matters: this cannot live only as a convention in application code that individual developers are expected to remember, it needs a single, shared function, `CanTransition(currentState, requestedState)`, that every code path attempting a state change must call, returning a clear rejection if the transition is not in the allowed map. This turns "an order went backward" from a possible-but-rare bug into a structurally rejected operation, the same discipline this course applies to preventing invalid cross-domain access.

**Key principle.** *A state machine's legal transitions must be enforced in one shared, unavoidable place, never left to the discipline of whoever is writing a given code path that day.*

## Q17. The double-tap problem

**Scenario.** A customer on a slow mobile network taps "Place Order," sees no immediate response, and taps it again three seconds later. Both requests reach the server.

**Question.** What happens, and how do you prevent it?

**Answer.** Without protection, this creates two separate, fully valid orders, each independently charged, because from the server's point of view these are simply two ordinary, unrelated `POST /orders` requests carrying identical content. The fix is an **idempotency key**: the client generates a unique key (typically a

UUID) once, at the moment the user first taps "Place Order," and includes it as a header on the request, resending the *same* key on any retry, including this double-tap scenario. The server persists this key, alongside the resulting order, under a unique database constraint the moment the first request is processed. When the second, duplicate request arrives carrying the same key, the server recognizes it, does not create a second order, and simply returns the result of the original request. Critically, this check-and-create must happen atomically, in the same database transaction, or a narrow race window reopens the exact problem the mechanism exists to close.

**Key principle.** *Any client action a user might plausibly repeat by accident needs an idempotency key; a "please don't double-click" warning on the button is not a real fix.*

## Q18. The lost update problem

**Scenario.** A restaurant's app and Tadka's own system both attempt to update the same order's status at nearly the same instant, one to Preparing, one to Cancelled (a customer cancellation racing a restaurant confirmation). Whichever write lands last in the database silently wins, with no error.

**Question.** Why is this dangerous, and how do you prevent it?

**Answer.** This is a classic lost-update race: both processes read the order in its prior state, both decide on a new state based on that stale read, and the database, having no idea a conflict occurred, simply applies whichever write arrives second, discarding the first write with no warning to either caller. The standard fix is **optimistic concurrency control**: every row carries a version marker, either an explicit version integer column or PostgreSQL's own built-in `xmin` system column, and every update is made conditional on that version still matching what was read: `UPDATE orders SET status = 'Cancelled', version = version + 1 WHERE id = 101 AND version = 5`. If the restaurant's update already changed the version, this update affects zero rows, and the application detects that as a conflict, re-reads the current state, and decides how to resolve it (reject the cancellation because the order is already being prepared, for instance), rather than silently overwriting a change nobody knew had happened.

**Key principle.** *A concurrent write should be detected and handled explicitly, never silently discarded by whichever transaction happens to commit last.*

## Q19. BookMyShow-style oversell, solved with Postgres `xmin`

**Scenario.** Tadka runs a flash promotion: "only 1 free dessert available per restaurant per hour." Two customers at the same restaurant tap "claim" within the same 50 milliseconds.

**Question.** Walk through, in detail, how `xmin`-based optimistic locking prevents both from succeeding.

**Answer.** Both requests first `SELECT` the promotion row, each independently seeing `remaining = 1` and each independently reading that row's current `xmin` value (an internal Postgres column that changes every time the row is updated, making it usable as a natural version marker without adding a dedicated column). Both requests then attempt `UPDATE promotions SET remaining = remaining - 1 WHERE id = 7 AND xmin = :capturedXmin`. PostgreSQL processes these two `UPDATE` statements one after another, never truly simultaneously at the storage engine level; whichever one is processed first succeeds, actually decrementing `remaining` to zero and, as a side effect of that successful update, changing the row's `xmin` to a new value. The second `UPDATE`, still carrying the *original* `xmin` it captured before either write happened, now finds zero matching rows, because the `WHERE` clause's `xmin` condition no longer matches, and the application correctly reports "sorry, that offer just ran out" to the second customer, exactly the

oversell prevention a ticket-booking platform like BookMyShow needs for its last available seat.

**Key principle.** *Optimistic locking does not prevent two requests from racing; it guarantees that only one of them can win, and makes the other one detectably, safely fail.*

## Q20. Ordering a side effect (SMS) relative to a database commit

**Scenario.** Tadka's order-confirmation code sends a confirmation SMS immediately after saving the order to the database, but before the transaction has actually committed. The SMS provider is instant; PostgreSQL's commit occasionally takes a moment longer under load.

**Question.** What can go wrong, and how do you fix the ordering?

**Answer.** If the SMS is sent before the transaction commits, there is a real window where the customer receives "your order is confirmed" while the actual commit has not yet happened, and if the transaction subsequently fails or is rolled back for any reason (a constraint violation, a deadlock retry that ultimately aborts), the customer has been told about an order that does not actually exist in the database. The fix is straightforward in principle: side effects that inform the outside world must happen *after* the transaction that creates the underlying fact has durably committed, never before, and ideally not even in the same request-handling code path at all, but via a separate step (or, more robustly, the outbox pattern) that is triggered specifically by a confirmed commit. The deeper lesson is the general shape of this whole class of bug: any communication with something outside your own database, an SMS, an email, a webhook to a partner, is not something a database transaction can roll back, so it must never be allowed to happen speculatively, before you are certain the fact it is describing is actually true and durable.

**Key principle.** *Never tell the outside world about something your own database has not yet durably confirmed.*

## Q21. Why should the API never leak internal exception details?

**Scenario.** A bug causes `POST /orders` to return `{"error": "NullPointerException at OrderService.cs line 142, connection string: Server=prod-db-01;..."}`.

**Question.** What is wrong with this, beyond it simply looking unprofessional?

**Answer.** This is a genuine security incident, not merely a cosmetic issue: it discloses internal file structure, exact line numbers (useful to an attacker probing for known vulnerabilities in a specific code version), and, in the worst case shown here, connection details that should never leave the server under any circumstance. The fix is a global exception-handling layer that catches every unhandled exception at the outermost boundary of the application, logs the full, detailed internal error to the server's own logging and monitoring system (where developers can see it), and returns a deliberately generic, safe error to the client, using the same consistent error envelope from Q13, with a correlation or trace ID the client can quote back to support, letting engineers look up the real, detailed error internally without ever exposing it externally. Detailed diagnostic information is valuable; it simply belongs exclusively on the internal side of the boundary between your system and the outside world.

**Key principle.** *Detailed errors belong in your logs, for your engineers; the client gets a generic, safe message and a trace ID.*

## Q22. Where should "restaurant is closed" be validated?

**Scenario.** Tadka needs to reject an order if the target restaurant is currently closed. One engineer wants to check this in the API controller, another wants a database constraint, a third wants it deep in the domain service.

**Question.** Where does this validation actually belong?

**Answer.** This is a genuine business rule about domain state, so its authoritative home is the domain service layer, specifically inside the `Order` creation logic that already has to load the restaurant's current state to build the order in the first place; this is where the rule can be expressed clearly, tested in isolation, and kept consistent regardless of which API surface eventually calls it (the same rule must apply whether the order comes from the mobile app, a future partner API, or an internal admin tool). A lightweight version of the same check can also live at the API controller layer purely as an early, cheap rejection for obviously bad requests, but that should never be treated as the actual source of truth. A hard database constraint is the wrong tool here specifically, because "is the restaurant open" is a time-varying, business-logic-dependent condition (opening hours, holiday overrides, a manual "temporarily paused" toggle), not a simple structural invariant like "email must be unique," which is exactly the kind of rule a `CHECK` constraint is built for.

**Key principle.** *Structural invariants belong in the database; time-varying business rules belong in the domain layer, checked at the one place that is guaranteed to run regardless of which client is calling.*

## Q23. Implementing an idempotency key end to end

**Scenario.** You are asked to actually implement the idempotency mechanism from Q17 for `POST /orders`, not just describe it conceptually.

**Question.** Describe the concrete database schema and request flow.

**Answer.** Create a table, `idempotency_keys` (`key` TEXT PRIMARY KEY, `order_id` BIGINT NOT NULL, `response_body` JSONB NOT NULL, `created_at` TIMESTAMPTZ NOT NULL DEFAULT `now()`), with the primary key constraint doing the actual uniqueness enforcement. The request flow: the client sends `Idempotency-Key: <uuid>` as a header; the server, inside a single database transaction, first attempts `INSERT INTO idempotency_keys (key, order_id, response_body) VALUES (...)` using a placeholder, or checks for an existing row with that key before proceeding; if a row already exists, the server immediately returns the stored `response_body` without re-running any order-creation logic at all; if no row exists, the server creates the order, builds the response, inserts the idempotency record with the real `order_id` and `response_body`, and commits everything together in one transaction. The `INSERT`'s primary-key uniqueness constraint is what makes this safe even if two identical requests race each other at the database level: one insert succeeds, the other fails on the constraint violation, and the losing request's handler catches that specific failure and falls through to reading the winning request's stored response instead.

**Key principle.** *The uniqueness guarantee for an idempotency key must be enforced by the database itself, not merely checked by application code before an insert.*

## Q24. Why does a resource need a stable, immutable identifier?

**Scenario.** An engineer proposes using the customer-facing order number, like `TADKA-2026-004821`, directly as the database primary key, "since it's already unique anyway."

**Question.** What is wrong with this, and what should the design be instead?

**Answer.** A customer-facing identifier is subject to business-driven change in ways an internal primary key must never be: the format might change (a rebrand, a new numbering scheme for a new region), a support team might need to reissue a corrected reference number for a specific order after a dispute, or the display format might need localization. If that exact string is also the database primary key, every foreign key referencing it throughout the schema, every log line, every external system integration, becomes entangled with a value that business requirements can force to change. The correct design keeps them separate: an internal BIGINT or UUID primary key that never changes and is never shown to a customer, and a separate `order_reference` column, indexed for fast lookup, that holds the customer-facing string and can be regenerated or reformatted independently without touching a single foreign key anywhere in the system.

**Key principle.** *An identifier meant for external, human, or business use should never also be the internal key everything else is wired to.*

## Q25. An order stuck forever in "PaymentPending"

**Scenario.** A customer's order has been sitting in `PaymentPending` for six hours. The customer cannot re-order the same items easily, and support has no clear way to resolve it.

**Question.** What does this reveal about the state machine design, and how should it be fixed?

**Answer.** This reveals a state machine with a gap: it defined how to enter `PaymentPending`, but never explicitly defined what happens if the pending state never resolves, treating the "eventually" in "payment will eventually complete" as if it were guaranteed rather than merely likely. A properly designed state machine must plan for this explicitly: an automated timeout (say, thirty minutes) that transitions a still-pending order to a distinct `PaymentTimedOut` or `PaymentFailed` state, freeing the customer to retry cleanly, releasing any inventory or promotional holds the pending order was carrying, and giving support a clear, queryable state to search for and act on, rather than a `PaymentPending` order that is indistinguishable in the database from one that is genuinely thirty seconds old and progressing normally. The general lesson: **every state in a state machine needs an answer to "what happens if this state lasts forever," not just an answer to how you enter and successfully exit it.**

**Key principle.** *A state machine is incomplete until every state has an explicit answer for "what if this never resolves," not just for the happy path out of it.*

## Part 3

### Week 3, Architect for Scale, Scaling the Monolith

#### Q26. Query latency degrades from 50ms to 3 seconds as the table grows

**Scenario.** The `GET /orders?customerId=123` endpoint took 50ms when the `orders` table had 100,000 rows. Now, at 10 million rows, the same endpoint takes 3 seconds.

**Question.** What do you check, and in what order?

**Answer.** Resist the instinct to immediately add application servers or a read replica; neither fixes a genuinely slow query, it just runs the same slow query on more machines. First, run `EXPLAIN ANALYZE` on the actual query to see whether PostgreSQL is doing a sequential scan across all 10 million rows instead of using an index; if `customer_id` (and, if the query also filters or sorts by other columns, those too) is not properly indexed, this is very likely the entire problem, and a composite index matching the actual query shape can bring 3 seconds back down to milliseconds immediately, at zero infrastructure cost. Second, check the connection pool: if the application is exhausting its available database connections under load, requests queue waiting for a connection, which looks identical to "the query is slow" from the outside even though the query itself might be fast once it actually runs. Only after confirming the query is well-indexed and connections are not the bottleneck does it make sense to consider a read replica for genuinely read-heavy load, or caching for repeat reads of the same data. Adding infrastructure before this diagnosis is treating a symptom, not the disease.

**Key principle.** *Diagnose with `EXPLAIN ANALYZE` before reaching for more infrastructure; a missing index disguised as "we need to scale" is one of the most common and most avoidable production incidents.*

#### Q27. Why doesn't adding more app servers fix a database bottleneck?

**Scenario.** Under load, Tadka's API response times degrade badly. The on-call engineer's first instinct is to double the number of application server instances.

**Question.** Why might this not help, and in what specific case does it actively make things worse?

**Answer.** If the actual bottleneck is the database, CPU-bound on a bad query plan, or saturated on I/O, or simply out of available connections, adding more application server instances increases the number of clients competing for that same fixed, already-saturated database resource, without adding any database capacity at all. In the connection-exhaustion case specifically, this actively makes things worse: more application instances means more total connections being requested from the same database connection limit, potentially pushing an already-struggling database over the edge into outright connection refusals for everyone, a self-inflicted escalation caused by the "fix." The correct diagnostic habit, restated from the previous question, is to identify which specific resource is actually saturated, CPU, memory, disk I/O, network, or connection pool, on which specific tier, before touching capacity anywhere, because scaling the wrong tier is not merely unhelpful, it can be actively harmful.

**Key principle.** *Scaling the tier that is not the bottleneck does not help, and can make a connection-pool bottleneck actively worse.*

## Q28. The cache-aside pattern for the menu API, end to end

**Scenario.** You are asked to design caching for GET /restaurants/{id}/menu, currently a direct Postgres query on every request.

**Question.** Describe the full read and write flow of a cache-aside implementation.

**Answer.** On a read, the application first checks Redis for the key menu:{restaurantId}. If present (a cache hit), it is deserialized and returned immediately, without touching Postgres at all. If absent (a cache miss), the application queries Postgres for the current menu, stores the result in Redis under that same key with a reasonable TTL (a few minutes, since menus change infrequently), and then returns it to the caller. On a write, when a restaurant updates its menu, the application updates Postgres as the source of truth first, and then explicitly deletes (not updates) the corresponding menu:{restaurantId} Redis key, so that the very next read is guaranteed to be a cache miss that repopulates the cache with fresh data, rather than risking a stale or partially-updated cached copy. The TTL exists purely as a safety net, in case some write path ever fails to explicitly invalidate the cache, ensuring staleness never becomes permanent even if the explicit invalidation is missed once.

**Key principle.** *Cache-aside reads populate the cache on a miss; writes invalidate (delete) the cache entry, they do not try to cleverly update it in place.*

## Q29. "The cache lies": stale menu prices for ten minutes

**Scenario.** A restaurant updates a dish's price in the admin panel. Customers keep seeing the old price for the next ten minutes.

**Question.** Diagnose the likely cause and fix it.

**Answer.** The most likely cause, given the ten-minute window, is that the menu update code path only writes to Postgres and relies entirely on the cache's TTL to eventually expire the stale entry, rather than explicitly invalidating the Redis key the moment the price actually changes; if the TTL happens to be set to ten minutes, this exactly matches the observed symptom. The fix is precisely the write path described in Q28: the moment the admin panel's price-update endpoint successfully commits the change to Postgres, that same request handler must also issue a DEL menu:{restaurantId} against Redis, in the same logical operation, so staleness is bounded by "until the next read after this write," not by "until the TTL expires," which is a meaningfully weaker and slower guarantee for something as customer-visible and trust-sensitive as displayed pricing.

**Key principle.** *If a cache's only invalidation mechanism is its TTL, its staleness window is exactly the TTL, no better; explicit invalidation on write is what actually keeps a cache honest.*

## Q30. Live order tracking without polling

**Scenario.** Tadka's live delivery-tracking map currently works by having the mobile app call GET /orders/{id}/location every three seconds.

**Question.** Compare polling, long polling, Server-Sent Events, WebSockets, and Redis Pub/Sub for this feature, and pick one.

**Answer.** Plain polling every three seconds wastes enormous request volume on "nothing has changed" responses and adds a structural three-second worst-case delay before a real update is even seen. Long polling holds each request open until an update is available, reducing wasted requests but adding real

connection-management complexity on the server for very large numbers of concurrently open connections. WebSockets give full bidirectional, low-latency communication, genuinely powerful, but heavier than this feature needs, since the client only ever needs to *receive* location updates, never send anything back over the same channel. Server-Sent Events (SSE) fit this specific use case well: a single, simple, one-directional HTTP connection the server can push updates over as they happen, with automatic client-side reconnection built into the browser and mobile HTTP client behavior, and comfortably supported by standard infrastructure without needing a separate WebSocket-aware load-balancing setup. On the backend, the delivery partner's location updates are published to a Redis Pub/Sub channel keyed by order ID, and the SSE connection for that specific order simply subscribes to that channel and forwards whatever arrives straight to the connected client.

**Key principle.** *Choose the transport based on whether communication is one-directional or bidirectional, not by defaulting to whichever technology sounds most modern.*

### Q31. The hot key problem: a restaurant goes viral

**Scenario.** A Tadka restaurant is featured in a viral Instagram reel. Its `menu:{restaurantId}` Redis key jumps from baseline traffic to 40,000 reads per second, while every other key stays at normal levels.

**Question.** What actually breaks, and how do you fix it?

**Answer.** In a sharded Redis topology (Cluster, or client-side sharding), a single key always lives on exactly one node, so all 40,000 reads per second land on that one node regardless of how many other nodes exist in the cluster; that node's CPU and network saturate, its latency degrades badly, and because the rest of the cluster looks perfectly healthy on aggregate dashboards, this specific failure is often confusing to diagnose at first. The most effective fix is adding a very short-lived (a few seconds) local, in-process cache in the application layer itself, in front of Redis, so that the overwhelming majority of reads for this one hot key never even reach Redis; this removes load rather than merely redistributing it. A secondary mitigation is key splitting, storing the same value under several keys (`menu:{restaurantId}:0` through `:9`) and having clients pick one at random for reads, spreading the load across multiple nodes at the cost of needing writes to update every copy and tolerating brief staleness between them.

**Key principle.** *A hot key is a distribution problem, not a capacity problem; adding more nodes to the cluster does not help if one specific key still only ever lives on one of them.*

### Q32. Cache stampede at exactly 12:00 noon

**Scenario.** Five thousand identical menu-cache keys, all populated around the same time the previous day, all expire within the same one-second window at noon. All five thousand requests miss simultaneously and hit Postgres at once.

**Question.** What is this called, and what are the fixes?

**Answer.** This is a **cache stampede**, or **thundering herd**: a routine, synchronized cache expiry converts into a sudden, unplanned burst of database load the database was never sized to absorb all at once. The cheapest, always-worth-doing fix is TTL jitter: adding a small random offset to each key's expiry (`300 + random(0, 30)` seconds) so that keys populated around the same original moment do not all expire in the same instant. A stronger fix for genuinely hot, expensive-to-recompute keys is a single-flight lock: the first request to miss acquires a short-lived lock and is the only one that actually queries Postgres, while

every other concurrent request either waits briefly for that result or is served the previous, now-expired value until the rebuild completes. Stale-while-revalidate is a third option, serving the expired value immediately to every request while refreshing it in the background, appropriate wherever a few seconds of staleness is genuinely invisible to the user, which describes most of what a menu cache actually holds.

**Key principle.** *A synchronized cache expiry is a self-inflicted traffic spike; jitter the TTLs so keys never expire in a single, coordinated instant.*

### Q33. Connection pool exhaustion during a flash sale

**Scenario.** During a promotional flash sale, the application suddenly starts throwing "too many connections" or "connection pool exhausted" errors, even though the database server's CPU utilization looks only moderately elevated.

**Question.** Diagnose the likely cause.

**Answer.** This pattern, connection errors with only moderate CPU usage, points specifically at the connection pool's configured size being too small for the actual concurrency the flash sale generated, or at connections being held open for longer than necessary somewhere in the request path (a slow external call, like a payment gateway request, made while a database connection is still checked out and idle, waiting). The fix has two parts: first, size the connection pool deliberately based on actual concurrency needs and the database's own maximum connection limit, not an arbitrary default value nobody has revisited since the project started; second, and more importantly, audit the code for any place a database connection is held open across a slow, unrelated operation (exactly the anti-pattern flagged in the concurrency-control material: never hold a database connection, or a transaction, open across an external network call), releasing connections back to the pool the moment the database work they were needed for is actually done.

**Key principle.** *"Too many connections" with moderate database CPU usually means connections are being held too long, not that the database itself needs more raw capacity.*

### Q34. When do you actually need a read replica?

**Scenario.** A teammate proposes adding a read replica "just in case," even though the primary database's CPU is comfortably under 40% utilization.

**Question.** Should you add it, and what does adding a replica change about read-after-write behavior?

**Answer.** No, not yet; a read replica solves a capacity problem that does not currently exist at 40% CPU utilization, and adding one preemptively adds real, ongoing complexity (routing logic, monitoring at an additional node, replication-lag awareness) for a benefit that has no current evidence behind it. The moment a replica genuinely is warranted and added, it introduces a new behavior that must be explicitly designed for: reads sent to the replica are no longer guaranteed to reflect the very latest writes, because replication is asynchronous and takes real, nonzero time. A customer who places an order and is immediately shown their order history, if that history read goes to a lagging replica, might briefly see the order missing entirely. The standard fix is routing any read that immediately follows a write by the same actor to the primary specifically, for just that one read, while sending the large majority of reads that are not consistency-sensitive to the replica.

**Key principle.** *A read replica is not free capacity; it is a capacity increase that comes bundled with a new consistency question every read path must explicitly answer.*

### Q35. Redis is down for 30 minutes during peak

**Scenario.** Redis becomes completely unreachable for thirty minutes during Tadka's dinner-rush peak.

**Question.** What should degrade gracefully, and what should not be affected at all?

**Answer.** Anything relying on Redis purely as a cache in front of Postgres, the menu, restaurant ratings, should degrade gracefully via cache-aside's own fallback: every request simply falls through to Postgres directly, which is slower but still fully correct, exactly the behavior this course establishes for a correctly classified performance dependency. The genuinely dangerous part, and the one worth explicitly stress-testing before it happens for real, is the sudden, unmitigated shift of the *entire* cache-miss traffic volume onto Postgres all at once, at the worst possible moment, peak load; if that shift is large enough, it can turn a Redis outage into a second, unrelated Postgres outage through sheer overload, which is a materially worse incident than the original cache failure. Order data, payment state, and anything already correctly classified as living solely in Postgres should be entirely unaffected, since Redis was never their source of truth to begin with, precisely the value of having classified each use of Redis by its actual business role ahead of time, rather than discovering the classification live, during the incident.

**Key principle.** *A cache outage should degrade the cached feature's latency, not become a second outage in whatever sits behind the cache; that second failure is a capacity-planning gap, not an inevitable consequence of the cache failing.*

### Q36. Defending a scale number before choosing Redis, replicas, or sharding

**Scenario.** An interviewer challenges you: "you said Tadka needs Redis and a read replica. Prove it, with numbers, don't just assert it."

**Question.** How do you actually construct this argument?

**Answer.** Start from the defensible base number from Q7: roughly 1 lakh orders a day, averaging about 1.2 orders per second, with a dinner-rush peak in the neighborhood of 20 to 25 orders per second. Menu-browsing traffic is typically an order of magnitude or more above order-placement traffic, since most users browse several restaurants before ordering once; call it a conservative 200 to 300 menu reads per second at peak. A single, reasonably sized Postgres instance can handle the order-write volume comfortably on its own, which is exactly why sharding is not yet justified, there is no evidence of a write bottleneck at this volume. The 200 to 300 reads per second of largely repeated, rarely-changing menu data is precisely the shape of load a cache exists to absorb cheaply, which is the concrete, numeric justification for Redis specifically for this workload, not "because caching is generally good." A read replica becomes justified only once a *specific*, measured read path (order-history browsing, an analytics dashboard) is shown to be consuming a meaningful, evidenced share of the primary's capacity, which at this stated scale has not yet been demonstrated.

**Key principle.** *Every infrastructure recommendation should be traceable back to a specific number, applied to a specific access pattern, not asserted as a generically good practice.*

### Q37. Rate limiting the public order-placement endpoint

**Scenario.** A bot script discovers Tadka's `POST /orders` endpoint and begins placing thousands of fake orders per minute, aiming to abuse a "first order free" promotion.

**Question.** What rate-limiting approach would you apply, and why?

**Answer.** A token bucket algorithm, keyed per customer account (and, ideally, layered with a secondary limit per IP address or device fingerprint to catch account-cycling abuse), fits this well: it allows a normal customer's ordinary burst of activity (browsing, adding to cart, placing one order) without friction, while capping the *sustained* rate at which any single identity can place orders, which is exactly the abuse pattern being defended against. A strict fixed-window limiter would be weaker here, since a bot can burst right at a window boundary and still get through at nearly double the intended rate. Beyond the rate limiter itself, this specific scenario, an abused promotion, also needs a business-logic control that a rate limiter alone cannot provide: the "first order free" promotion's eligibility check needs its own idempotent, persistent record per customer identity (tied to a verified phone number or payment method, not merely an account that can be trivially recreated), because rate limiting slows down abuse, it does not, by itself, prevent a determined attacker from eventually succeeding at a slower pace.

**Key principle.** *Rate limiting controls the pace of abuse; it does not replace a correct, persistent eligibility check for anything that must only ever happen once per genuine customer.*

### Q38. Why Redis before splitting into microservices?

**Scenario.** A stakeholder asks, "we're struggling with load, should we add Redis, or should we just split into microservices to scale better?"

**Question.** Which comes first, and why?

**Answer.** Redis, and by a wide margin, because it directly addresses the actual, most common bottleneck at this stage, repeated, expensive reads of largely unchanging data, at a small fraction of the operational cost a service split requires. Splitting into microservices is a structural, largely irreversible reorganization of the entire system, adding network calls, separate deployments, and distributed debugging, and it does not, by itself, make a single query faster or a single hot read cheaper; a slow, uncached menu query run from three separate microservices is simply the same slow, uncached query, now duplicated across a more complicated topology. The natural order this course has followed throughout is diagnosis first (find the actual bottleneck), then the cheapest fix that addresses it directly (an index, then a cache, then a read replica), and only after those are genuinely insufficient, or after a *specific*, evidenced organizational or fault-isolation reason appears, does splitting a service become the right next move, exactly the sequencing Tadka itself followed on the way to extracting Payment.

**Key principle.** *Reach for a cache before reaching for a network boundary; one is a targeted fix for a specific bottleneck, the other is a structural commitment with a much higher, harder-to-reverse cost.*

## Part 4

### Week 4, Modular Monolith and First Service Extraction

#### Q39. What does a module boundary actually look like in code?

**Scenario.** Tadka's codebase has folders named `Orders/`, `Payment/`, `Restaurant/`, but a code review reveals `Orders/OrderService.cs` directly instantiating and calling classes from `Payment/PaymentProcessor.cs`.

**Question.** Is this a real module boundary? If not, what would make it one?

**Answer.** No, this is a folder convention, not a boundary; nothing in the current setup actually prevents `Orders` code from reaching into `Payment`'s internals, and a folder structure that looks clean in a diagram provides zero real protection against that coupling deepening over time under deadline pressure. A real module boundary requires two things: an explicit, narrow interface that is the *only* sanctioned way for one module to interact with another (`Orders` should depend on something like `IPaymentClient`, never on `PaymentProcessor` directly), and an enforcement mechanism that actually fails when violated, most robustly a compile-time or CI-enforced architecture test (using a tool like `NetArchTest` in .NET or `ArchUnit` in Java) that fails the build the instant `Orders` code references `Payment`'s internal namespace, turning "please don't do this" from a code-review convention people can forget under pressure into a build failure nobody can ship past.

**Key principle.** *A module boundary that is not enforced by a failing build is a naming convention, not a boundary; the enforcement is what makes it real.*

#### Q40. The four questions before extracting Restaurant into its own service

**Scenario.** Following `Payment`'s successful extraction, a teammate proposes, "let's extract `Restaurant` next too, while we have the momentum."

**Question.** Apply the four-question test fresh to this specific proposal.

**Answer.** First, what problem are we actually solving: name it specifically for `Restaurant`, not by analogy to `Payment` ("Payment needed it, so `Restaurant` probably does too" is not evidence, it is momentum). Second, does that problem exist today, with real data: is `Restaurant`'s traffic profile actually different from the rest of the system, does it hold data with a stricter security requirement than an order or a menu item, is a different team genuinely blocked by shared deployment; if none of these are true yet, there is no current problem to solve. Third, can the monolith solve whatever problem does exist without a network boundary: a clean internal module boundary, enforced the way Q39 describes, often delivers most of the benefit people are actually asking for (independent reasoning, clear ownership) without paying the network-call cost. Fourth, what new failure modes would extraction introduce specifically for `Restaurant`, and are they smaller than the problem being solved: menu lookups are extremely read-heavy and latency-sensitive for the customer-facing browsing experience, so a new network hop here has a directly customer-visible cost that `Payment`'s less latency-sensitive, already-asynchronous flow did not carry in the same way. Extracting "because we have momentum" fails this test on the very first question.

**Key principle.** *Momentum from one successful extraction is not evidence for the next one; run the same four questions fresh, every single time, for every module.*

## Q41. Why Payment first, not Orders or Restaurant?

**Scenario.** An interviewer asks specifically why Payment, out of every module in Tadka's monolith, was the first one extracted.

**Question.** Give the specific, evidenced reasons, not a general statement about payment being "important."

**Answer.** Payment had the strongest possible case on exactly the two legitimate drivers this course treats as sufficient on their own: fault isolation and a security or compliance boundary. On fault isolation, a catastrophic failure inside Payment's code, an unhandled exception, a memory leak, has the structural potential to crash the entire shared process, taking Orders and Restaurant down with it even though they have nothing to do with the failure; a separate process makes this structurally impossible rather than merely unlikely. On the compliance boundary, Payment logic sits closest to card data, gateway credentials, and financial transaction records, categories of data that carry meaningfully stricter access, auditing, and retention obligations than a menu item or a delivery address, and a separate service with its own database materially shrinks who and what can touch that data. Orders and Restaurant, at Tadka's evidenced scale, have neither of these pressures yet: their failure does not carry outsized blast radius relative to the rest of the system, and they do not hold data requiring a stricter boundary, which is exactly why they correctly remained inside the monolith.

**Key principle.** *The strongest extraction candidate is the module with the clearest fault-isolation or compliance case, not the module that happens to feel the most "core" to the business.*

## Q42. Proving fault isolation after extracting Payment

**Scenario.** Payment has just been extracted into its own service. A teammate claims "we now have fault isolation," pointing at the architecture diagram.

**Question.** How do you actually verify this claim rather than accepting the diagram at face value?

**Answer.** Deliberately stop the Payment service and run three specific checks. `GET /menu` should return `200 OK` unaffected, since menu browsing has no dependency on Payment whatsoever; if it fails, something is secretly coupling them (a shared health-check dependency is the most common culprit). `GET /orders` for existing orders should also return `200 OK`, since that data lives entirely in Order's own database and requires no live call to Payment. `POST /orders`, creating a brand-new order, should still return `201 Created` with the order marked `PaymentPending`, provided payment processing was already moved off the synchronous request path *before* the extraction happened, which is exactly why the sequencing of decisions matters: extracting Payment while it was still called synchronously would have made this third test fail immediately, turning a Payment outage directly into an Order-placement outage. Only if all three checks genuinely pass has fault isolation been proven; a diagram showing two separate boxes is a claim, not evidence.

**Key principle.** *Fault isolation is proven by deliberately breaking the dependency and checking specific, named behaviors, not by pointing at a diagram showing two separate boxes.*

## Q43. Why HTTP now, Kafka deliberately later

**Scenario.** A teammate argues, "if we're doing microservices properly, Order and Payment should communicate through Kafka, not a plain HTTP call."

**Question.** Why did Tadka choose HTTP for this specific interaction, with Kafka deliberately deferred?

**Answer.** Kafka earns its operational cost, a broker to run and monitor, topics, partitions, consumer groups, offset management, its own retry and dead-letter strategy, once there is a genuine requirement for durable asynchronous delivery, high sustained throughput, multiple independent consumers of the same event, or message replay. At the point Payment was extracted, Order's interaction with Payment was a single, direct request with a single, direct consumer, exactly the shape a plain HTTP call is built for, and adopting Kafka at that moment would have added an entire category of new operational surface (broker health, consumer lag, poison messages) in exchange for a durability guarantee the system did not yet need enough to justify the cost. The deliberate sequencing matters pedagogically as much as practically: choosing infrastructure because a diagram looks more "properly distributed" with it included is precisely the mistake this course spends its entire fourth week correcting; Kafka becomes the right choice the moment a concrete, evidenced requirement for it actually shows up, which the course explicitly defers to a later stage rather than adopting preemptively.

**Key principle.** *Adopt messaging infrastructure because a specific requirement (durability, multiple consumers, replay) demands it, not because a synchronous call feels architecturally unfashionable.*

## Q44. What PCI compliance actually requires, versus what a service boundary provides

**Scenario.** After extracting Payment into its own service and database, a stakeholder announces, "great, we're PCI compliant now."

**Question.** Is this true? What has the extraction actually achieved, and what is still missing?

**Answer.** The extraction is a genuinely valuable step, but it is not, by itself, compliance. A separate service and database for Payment meaningfully shrinks the *scope* of systems that touch sensitive payment data, fewer engineers, fewer code paths, fewer databases need to be in scope for a PCI audit at all, which is a real and useful reduction in audit surface and risk. What it does not automatically provide: encryption of card data at rest and in transit (a specific, separate technical control), tokenization so that raw card numbers are never stored at all if avoidable, strict, audited access controls and logging on who can query the Payment database and when, network segmentation isolating the Payment service's infrastructure, and the actual documented policies, vulnerability scanning, and audit process PCI DSS requires as a certification, none of which a service boundary alone implements. The honest statement is "extracting Payment reduced our compliance scope and gave us a boundary to build the remaining controls around," not "we are compliant."

**Key principle.** *A service boundary shrinks compliance scope; it is the foundation the remaining required controls are built on top of, not a substitute for them.*

## Q45. Why sharing one Postgres instance between Order and Payment breaks the boundary

**Scenario.** To save on infrastructure cost, the newly extracted Payment service is pointed at the same Postgres instance Order Service already uses, just a different schema.

**Question.** Does this preserve the extraction's intended boundary? What specifically breaks?

**Answer.** Running both services against the same Postgres instance, even in separate schemas, can be an acceptable interim step *if* database-level permissions genuinely prevent cross-schema access (the schema-per-domain enforcement from Q5), but it is a meaningfully weaker boundary than fully separate database instances, and it directly undermines two of the specific reasons Payment was extracted in the first place. On fault isolation: a single Postgres instance now has two service's worth of load and lock contention on it, so a runaway query or a connection-pool exhaustion incident originating from Order can degrade Payment's database performance too, and vice versa, the exact shared-fate risk the process-level extraction was meant to eliminate. On the compliance boundary: an audit scope reduction is far weaker if Payment's data sits on the same physical database server as everything else, since infrastructure-level access controls, backup policies, and physical/network isolation are generally scoped per database instance, not merely per schema within one. The correct target state is fully separate database instances, with the same-instance, separate-schema setup treated explicitly as a temporary, cost-driven interim step, not the final architecture.

**Key principle.** *A shared physical database undermines both fault isolation and compliance scope reduction, even when logical schemas are kept separate; treat it as a temporary step, not the destination.*

## Q46. WAL and crash recovery: does a confirmed order survive a mid-transaction crash?

**Scenario.** A customer's order is confirmed, and the API returns `201 Created With status: Confirmed`. At that exact instant, the Postgres server hosting Tadka's database loses power.

**Question.** Will this order still exist, correctly, when the database comes back online?

**Answer.** Yes, provided the `201 Created` response was only sent *after* PostgreSQL had durably confirmed the commit, which is precisely what the API code must wait for before responding. PostgreSQL's Write-Ahead Log guarantees that once a transaction's commit record has been flushed to durable storage, that transaction is safely recoverable even if the actual table's data pages had not yet been separately written to disk at the moment of the crash; on restart, PostgreSQL replays the WAL from the last checkpoint forward and reconstructs exactly this change. The failure case worth being explicit about is the reverse ordering bug: if the application code somehow returned `201 Created` to the client *before* actually waiting for the database driver to confirm the commit succeeded (a bug in `async/await` handling, for instance, or a premature response sent based on an in-memory state change rather than a confirmed database acknowledgment), the customer could be told "confirmed" for an order that a crash at that exact moment would genuinely lose, because the durability guarantee was never actually obtained before the promise was made.

**Key principle.** *A durability guarantee is only real once you have actually received confirmation of it; never respond to a client based on an assumed or in-flight database write.*

## Q47. "Order not found" immediately after placing it

**Scenario.** A customer places an order, the app immediately calls `GET /orders/{id}` to show the confirmation screen, and occasionally receives a `404 Not Found`, even though the order was genuinely created successfully.

**Question.** Diagnose the cause and fix it.

**Answer.** This is the read-after-write consistency gap from Q34, made concrete: the confirmation-screen read is very likely being routed to a read replica that has not yet replayed the WAL for the order-creation write, which committed successfully on the primary a moment earlier but has not yet propagated across the (inherently asynchronous, by default) replication stream. The fix is routing this one specific read, the read that immediately follows this specific write, to the primary database rather than the replica, while leaving every other, non-immediately-following read free to use the replica as normal. A more general, though heavier, fix is configuring PostgreSQL for synchronous replication for order-related writes specifically, so the primary does not consider the write complete until the replica has confirmed receiving it, closing the gap entirely at the cost of added write latency on every order.

**Key principle.** *A confusing "not found immediately after creation" symptom is the signature of a read-after-write consistency gap; route that specific read to the primary rather than redesigning the whole read path.*

## Q48. Two minutes of confirmed orders missing after a failover

**Scenario.** The primary Postgres instance fails, a standby is promoted to the new primary, and afterward, two minutes' worth of orders that customers were shown as successfully confirmed are missing entirely from the new primary.

**Question.** What went wrong, architecturally, and what setting would have prevented it?

**Answer.** This is the asynchronous-replication data-loss window made real: under PostgreSQL's default asynchronous replication, the primary commits a transaction and tells the client "success" without waiting for the standby to have received that transaction's WAL at all, so any transactions committed in the window between the standby's last-received WAL position and the primary's actual failure are durably recorded only on the now-dead primary, and are simply absent from the promoted standby's history, with no way to recover them after the fact. The setting that would have prevented this is `synchronous_commit = on` combined with an appropriately configured `synchronous_standby_names`, which makes the primary wait for the standby to durably flush each transaction's WAL before acknowledging it to the client; this closes the data-loss window entirely, at the direct, unavoidable cost of added latency on every single write, since every commit now waits on a network round trip to the standby.

**Key principle.** *Under asynchronous replication, "the client was told it succeeded" and "the replica has this transaction" are two different guarantees; only synchronous replication makes them the same guarantee, at a real latency cost.*

## Q49. Scaling to 50 read replicas: direct fan-out versus cascading

**Scenario.** Tadka's read traffic has grown enormously, and the team now runs 50 read replicas, all connected directly to the primary.

**Question.** What problem does this create, how does cascading replication help, and what new risk does it introduce?

**Answer.** With 50 replicas connected directly, the primary is running 50 separate WAL-sender processes and 50 separate network streams purely for the job of replicating its own write-ahead log outward, which is real, measurable load on the primary that has nothing to do with serving the application's actual write traffic, and which grows linearly with every additional replica added. Cascading replication restructures this: the primary streams WAL to only a handful of replicas directly, and each of those, in turn, forwards

the same WAL it received onward to its own further downstream replicas, reducing the primary's direct connection count dramatically while every replica, at every level, still eventually receives the full stream. The new risk this introduces is a dependency chain that did not exist in the direct topology: if an intermediate replica goes down, every replica cascading from it loses its WAL feed simultaneously and falls behind, even though the primary itself, and every other independent branch of the topology, remains completely healthy, a failure mode that requires its own monitoring at every level of the cascade, not just at the primary.

**Key principle.** *Cascading replication trades primary-side fan-out load for a new failure dependency between replication tiers; it is worth reaching for once fan-out is a measured, evidenced bottleneck, not before.*

## Q50. The durability wound: a lost in-memory payment job after extraction

**Scenario.** Payment Service goes down for twenty minutes. During that window, Order Service continues accepting orders, marking each one `PaymentPending`, and queuing a background job in the application's own memory to attempt payment once Payment Service recovers. The worker process handling this queue restarts (a routine deploy) partway through the outage.

**Question.** What happens to the orders whose payment jobs were queued but not yet attempted, and what is the architectural lesson?

**Answer.** Every payment job that existed only in that worker process's memory at the moment of the restart is gone, regardless of whether Payment Service itself was healthy again by the time the worker came back online; the orders remain permanently stuck in `PaymentPending`, with nothing left in the system that remembers they still need payment attempted, until a customer complaint or a manual audit eventually surfaces the problem. The architectural lesson is precise and worth stating exactly: extracting Payment achieved genuine fault isolation, a Payment Service crash no longer takes Order Service's process down with it, but fault isolation is a guarantee about *space* (which process a failure is contained to), and it says nothing at all about *time* (whether work that must eventually happen actually survives the gap between "the dependency failed" and "the dependency recovered"). The fix is making that pending-work record durable, most simply by writing it into the database, in the same transaction that creates the order, rather than trusting it to a single process's memory, so a worker restart no longer erases work that has not yet been completed.

**Key principle.** *Separating two components in space (different processes) does not automatically make the work between them durable across time; that is a distinct requirement that has to be designed for on its own.*

## Closing note

Fifty questions is enough to notice the pattern repeating: almost none of these answers are actually about a specific technology. They are about the same handful of underlying questions, applied again and again, to a new scenario each time, what is the actual, evidenced problem, what does the simplest option cost, what new failure does the more complex option introduce, and what happens on the day this specific decision turns out to be wrong. A candidate who has internalized that pattern can answer a fifty-first question this document never asked, because the pattern, not the specific answer, is what an interview is actually testing for.